

Obiettivi

Capire come usare la
Programmazione Dinamica
PD

Optimizzazione -

INPUT:

OUTPUT: soluzione ammissibile
che ottimizza un costo
MINIMO / MASSIMO valore
relativo alla soluzione!

LCS una sottosequenza Z
comune a X e Y (input)
che è di massima
lunghezza $|Z|$ è MASSIMA!

• soluzione ottimale

una $LCS(X, Y) \rightarrow$ molte!

• ottimo

$|LCS(X, Y)|$ certo

ottimo e

UNICO!

Regole per applicare PD

• Definizione Ricorsiva di
come calcolare l'ottimo

Ricorrenza es. $i, j > 0$

$$c[i, j] = \begin{cases} 1 + c[i-1, j-1] \\ \max \{ c[i-1, j], c[i, j-1] \} \end{cases}$$

$\downarrow \quad \downarrow$
 $x_i \quad y_j$

Numero di sottoproblemi
di $\bar{e}$ polinomiale nella
dimensione dell'input -

$$\boxed{(n+1) \cdot (m+1)} \quad O(n \cdot m)$$

Regola principe per PD
sottostituisce attimo

↳ $\exists$ di una ricorrenza
per esprimere el attimo!
1 FASE

⇒ calcolo dell'attimo
bottom-up

2ª FASE

ricostruzione di una
soluzione!

costo della PD $\bar{e}$

legato al numero di
sottoproblemi (ottimi)

distinti da risolvere

Costo finale dipende

da quanti sono questi
sottoproblemi e quanto
costa risolvere il

singolo sottoproblema!

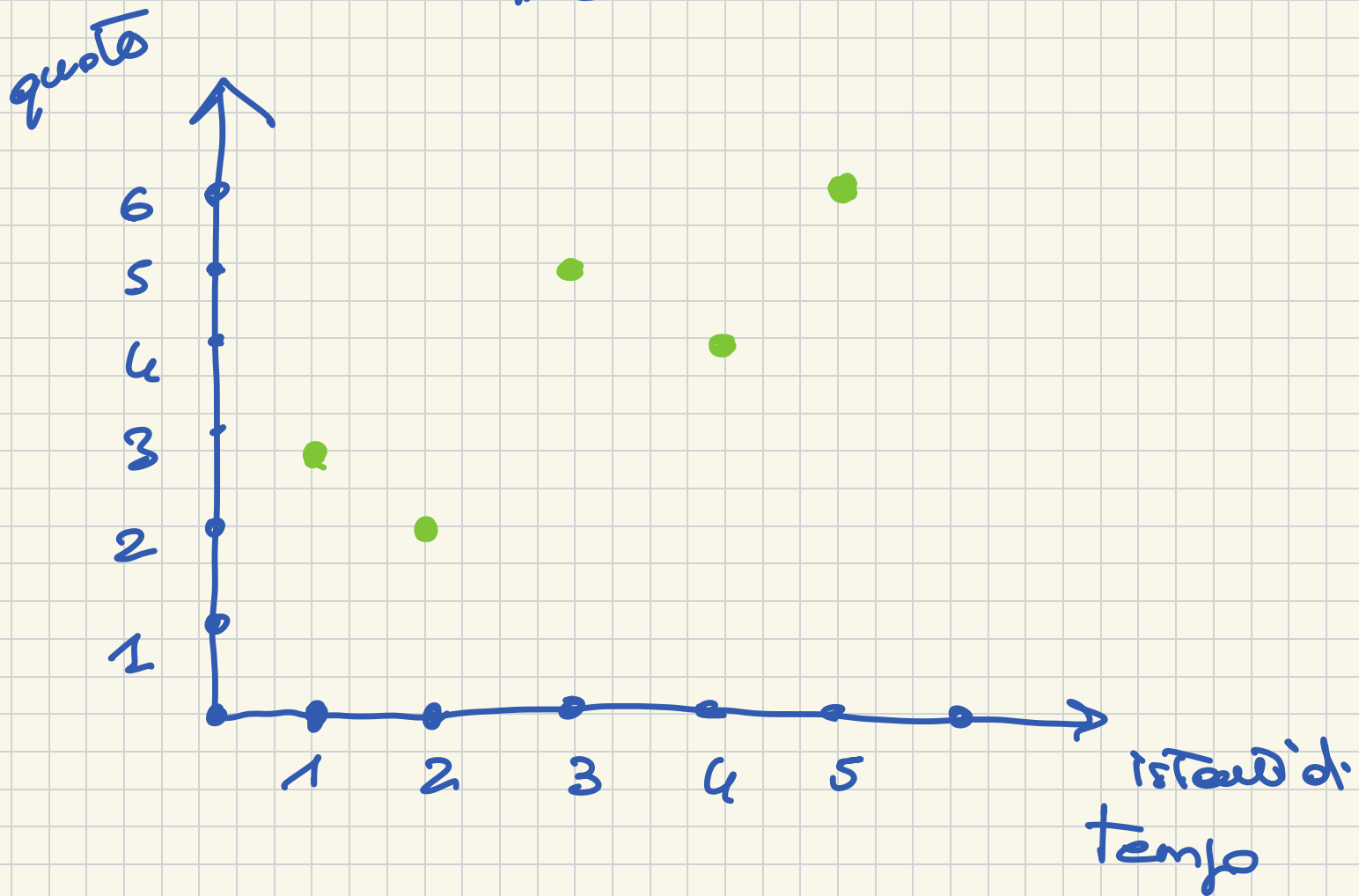
Es. | LCS | $(n+1) \cdot (m+1)$
sottoproblemi il singolo
problema ha costo $O(1)$

$$\Rightarrow O(n \cdot m) =$$

$$O(1) (n+1) \cdot (m+1)$$

Longest Increasing Subsequence

LIS



$$x_1 = 3$$

$$x_2 = 2$$

$$x_3 = 5$$

$$x_4 = 4$$

$$x_5 = 6$$

$\langle 3, 2, 5, 4, 6 \rangle$

sequenza quote
dei giorni

meccanismo di distruzione
dei "droni" che sono
e quote crescenti (rispetto
al progredire del tempo)

esempio data la
sequenza

$\langle 2, 1, 6, 7, 4, 5, 3 \rangle$

obiettivo
sotto sequenza di numeri
crescenti di massima
lunghezza

INPUT:

$$S = \langle x_1, x_2, x_3, x_4, \dots, x_k \rangle$$

$$\langle \textcircled{3}, 2, \textcircled{5}, 1, \textcircled{6}, 3, \textcircled{7}, 1 \rangle$$

x_{i_1} x_{i_2} x_{i_3} $x_{i_4} = x_{i_k}$

$$i_1 = 1$$

$$i_2 = 3$$

$$i_3 = 5$$

$$i_4 = 7$$

OUTPUT:

Tracce una più lunga
sotto sequenza dell'input, S
Tale che sia crescente!

DESCRIZIONE formale

input:

$$S = \langle x_1, x_2, \dots, \underline{x_n} \rangle$$

output:

$$Z = \langle x_{i_1}, x_{i_2}, \dots, \underline{x_{i_k}} \rangle$$

Tale che: $1 \leq i_1 < i_2 < i_3 \dots < i_k \leq n$

Z è sotto sequenza di S

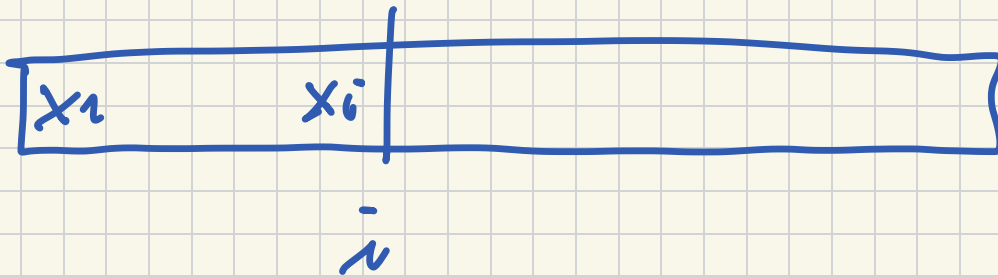
VINCOLO è che sia crescente!

$$x_{i1} < x_{i2} < x_{i3} \dots < x_{ik}$$

↓↓ z è crescente

obiettivo dell'ottimizzazione

$|z|$ è MASSIMA!



$n+1$

distinte di cui posso
calcolare la LIS
sotto sequenza

1° FASE della PD

calcolo dell'ottimo →

$|LIS(S)|$

$LIS(S)$

possibile

rappresenta una

LIS

di S

longest
increasing
subsequence

$|LIS(S)|$

$\Rightarrow$

lunghezza

ottimo

per
calcolare

$|LIS(S)|$

dev trovare una relazione

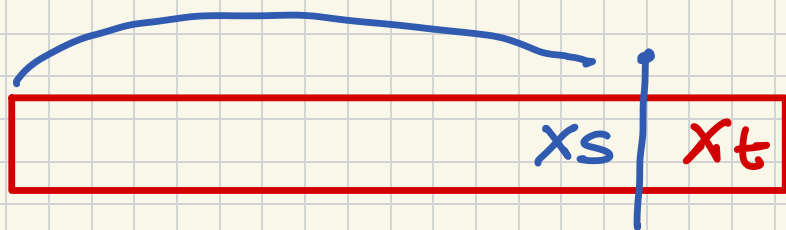
ricorsiva che esprime

l'ottimo in termini di

sotto ottimi! sottosoluzioni

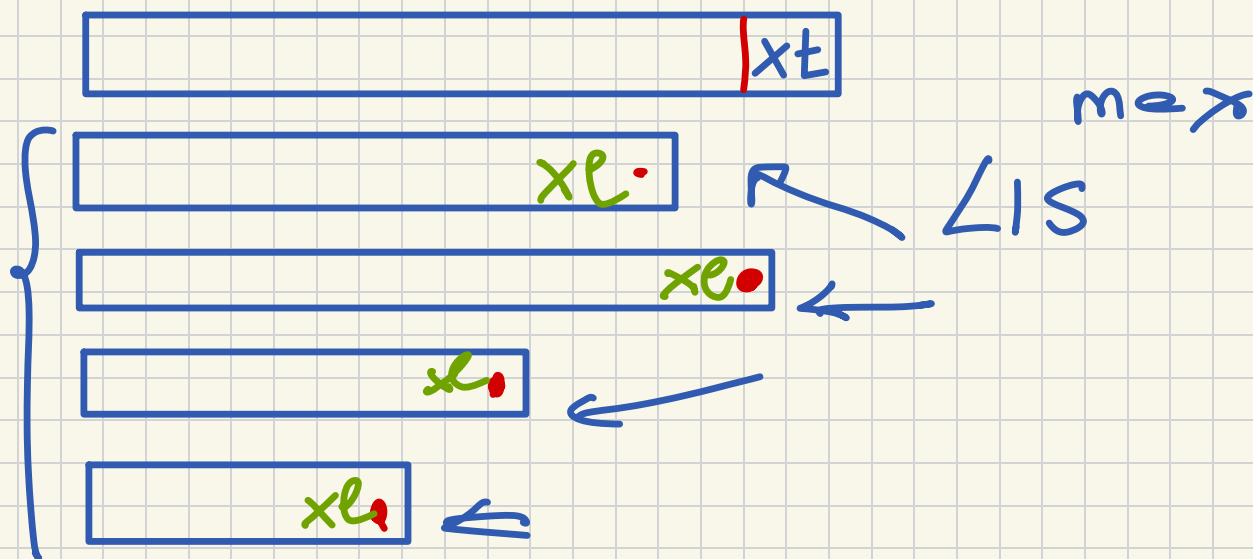
ottime

(prefissi di S !)



ottimo de finisce

un ottimo lo ottengo?
cerco de el ottimo sulle
LIS de finiscano tutte
prima di x_t / cioè
è la più lunga LIS
che termina con un valore
 x_l con $x_l < x_t$



Risolvo un problema
vincolato LIS_v

il vincolo è che calcolo
la LIS per la
sequenza $\langle x_1, x_2, \dots, x_i \rangle$
vincolate al fatto che

x_i sia parte finale
della LIS(S_i) -

calcolo LIS_v(S_i) $\forall 1 \leq i \leq n$

LIS(S) = la più lunga
tra le varie LIS_v(S_i) $\forall i$

$$S = \langle \underset{1}{1}, \underset{2}{18}, \underset{3}{4}, \underset{4}{15}, \underset{5}{6}, \underset{6}{9}, \underset{7}{7}, \underset{8}{16}, \underset{9}{8}, \underset{10}{3}, \underset{11}{2} \rangle$$

$$Z_1 = \langle 1 \rangle$$

n LIS_v

$$Z_2 = \langle 1, 18 \rangle$$

do
calculer!

$$Z_3 = \langle 1, 4 \rangle$$

$$Z_4 = \langle 1, 4, 15 \rangle$$

$$Z_5 = \langle 1, 4, 6 \rangle$$

$$Z_6 = \langle 1, 4, 6, 9 \rangle$$

$$Z_7 = \langle 1, 4, 6, 7 \rangle$$

$$Z_8 = \langle 1, 4, 6, 7, 16 \rangle \Rightarrow 5$$

$$Z_9 = \langle 1, 4, 6, 7, 8 \rangle \Rightarrow 5$$

$$Z_{10} = \langle 1, 4, 6, 7, 5 \rangle$$

$$Z_{11} = \langle 1, 2 \rangle$$

$$Z_9 = \langle 1, 4, 6, 7, 8 \rangle \text{ une LIS}(S)$$

$|LIS(S)|$ $\overline{e}$ ottimo
ottenuto

prendendo $\max_{1 \leq i \leq n} |LIS_{\vee}(S_i)|$

dobbiamo dire come
calcolare $LIS_{\vee}(S_i)$?

↓ sottostruttura ottima
per calcolare $|LIS_{\vee}(S_i)|$

$c_i =$ "lunghezza di
una $LIS_{\vee}$ che termina
con il simbolo x_i "

come calcolare c_i ?

CASO BASE

Ci posso ottenere
in modo "diretto"

INPUT:

$$S = \langle x_1 \rangle$$

risposta

$$LIS_v(S_1) = \langle x_1 \rangle$$

$$|LIS_v(S_1)| = 1$$

$$c_1 = 1$$

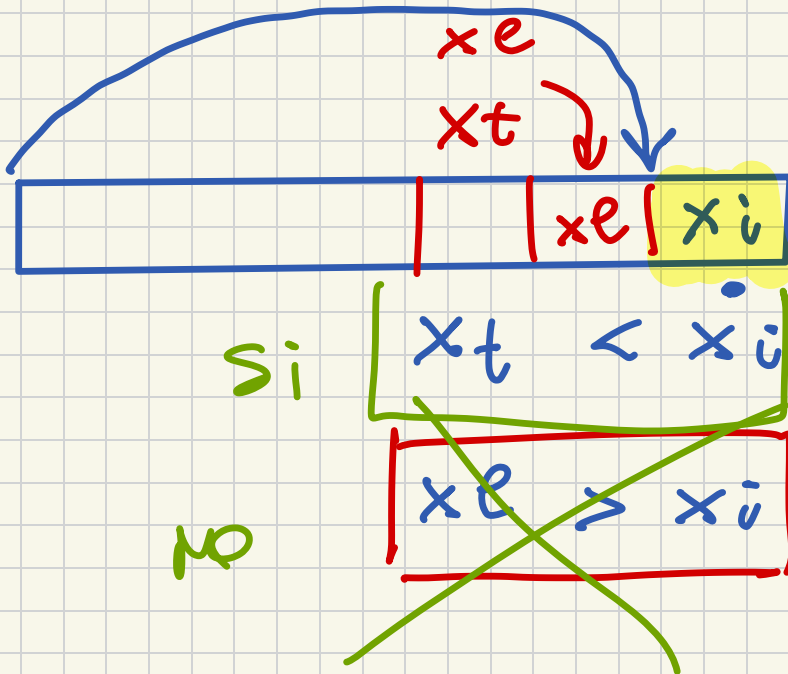
quindi

$$c_1 = 1$$

$$c_0 = 0$$

PASSO ricorsivo

$i > 1$

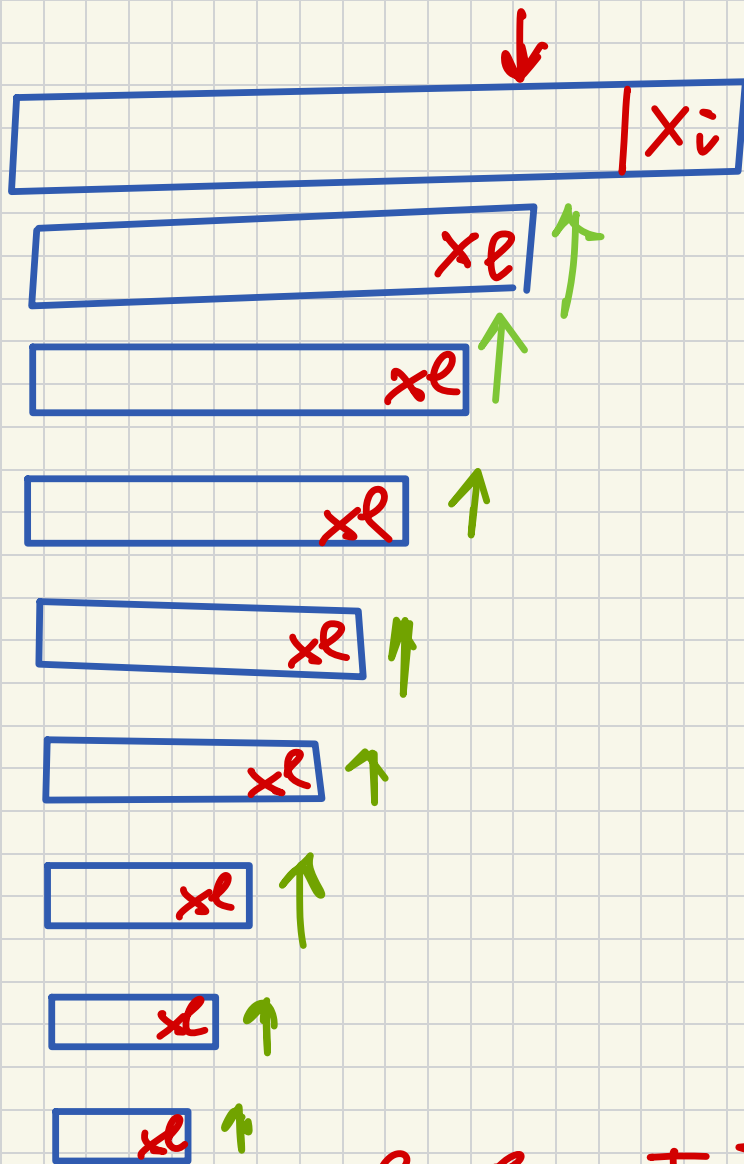


una
soluzione
ottimale
per
 $LIS(S_i)$
finisce con
 x_i

sotto sequenza delle
lis vincolate a finire con

un elemento che precede
 x_i ma è strettamente

$< x_i$



$$x_l < x_i$$

calcola tutte le

LIS $\forall$ per $S_j \quad \forall j < i$

tali che $x_j < x_i$

$$|LIS_v(S_i)| =$$

$$\Rightarrow \max_{\substack{j < i \\ x_j < x_i}} |LIS_v(S_j)| + 1 \quad (\text{relativo a } x_i)$$

$$c_i = \max_{+1} \left\{ c_j : x_j < x_i, \forall 1 \leq j < i \right\}$$

calcolare c_i dal basso
 per valori di $c_1, c_2, c_3 \dots$
 fino a c_i -

come si fa?

$S = \langle \underline{1, 18}, 4, 15, 6, 9, 7, 16, 8, 3, 2 \rangle$
 c_2

1	2	2									
1	2	3	4	5	6	7	8	9	10	11	

$c_1 \Rightarrow$ caso base

$c_2 \Rightarrow \max_{\text{vinced}} \{ c_1 \} + 1 = 2$
 $x_1 < x_2$

$c_3 = \max \{ c_1, \cancel{c_2} \} + 1 = 2$

$x_1 < x_3$? sì
 ~~$x_2 < x_3$?~~ No

$$S = \langle \underbrace{1}_{c_2}, 18, 4, 15, 6, 9, 7, 16, 8, 3, 2 \rangle$$

1	2	2	3	3	4	4	5	5	2	2
1	2	3	4	5	6	7	8	9	10	11

$$c_4 = \max \{ c_1, \cancel{c_2}, c_3 \} + 1 = 3$$

$$x_1 < x_4 \quad \text{si}$$

$$x_2 < x_4 \quad \text{no}$$

$$x_3 < x_4 \quad \text{si}$$

$$c_5 = \max \{ c_1, \cancel{c_2}, c_3, \cancel{c_4} \} + 1 = 3$$

$$c_6 = \max \{ \cancel{c_1}, \cancel{c_2}, c_3, c_5 \} + 1 = 4$$

$$c_{11} = 2 \neq \text{valore della } |LIS(S)|$$

$$|LIS(S)| = c_8 / c_9$$

$$c_1 = \max \{ \quad \} + 1 = 0 + 1$$

oppure

c_0 ?

• PSEUDO - CODICE per riempire il vettore
(REGOLA FURBA!)

• come ottengo $|LIS(S)|$
da LIS_v !

FORMULA :

$$d_n = \max \{ c_i : \forall 1 \leq i \leq n \}$$

$$\underbrace{|S| = n}_{\rightarrow |LIS_v(S_i)|}$$

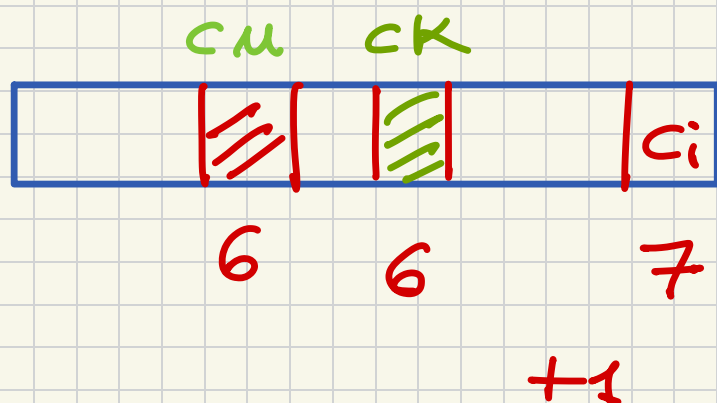
Pseudo - codice per
riempire il vettore C
dei coefficienti c_i -
per ottenere una soluzione
ammissibile per la 2^a FASE
(ricostruzione di una soluzione
ottimale) ho bisogno di
ricordare dove trovo il
massimo valore in C che
precede il valore c_i -

$$Lis_v(S_i)$$

la posizione di C dove
si trova il massimo valore
 $< c_i$ e sta alle
sinistre di c_i !

Per ricostruire una
soluzione per $LIS_{\sqrt{}}(S_i)$

devo tenere traccia di
dove si trova il valore
che precede c_i e che
uso per incrementare di 1
ed ottenere c_i



$K =$
parent
per c_i

$$\max \{ c_M, c_K, \dots \} + 1 = c_i = 7$$

$\downarrow \quad \downarrow$
6 6

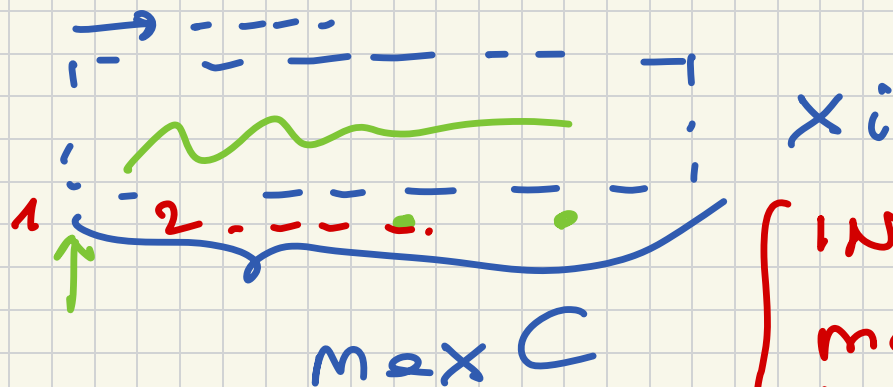
Pseudo-codice per
ricreare C e per
assegnare ad ogni posizione
 $i \rightarrow \text{parent}(i)$
 $\Downarrow$

serve a capire quale è
la posizione dove si trova
il valore che precede x_i
dove finisce $\text{LIS}()$
che poi finisce con x_i

per calcolare $ci \rightarrow$
iterazione su tutti i
valori che precedono i

- aggiornare il max valore
che trova a sinistra di i
tale che però è una

posizione da considerare



INIZIALIZ.
 $\max C = -1$
 $\text{parent}[i] = 0$

$x_h < x_i$

for $h = 1$ to $i-1$ do

if $x_h < x_i \wedge C[h] > \max C$
then

• $\max C = C[h]$

• $\text{parent} = h$

↓ serve a calcolare il
valore massimo precedente
e ci è la sua posizione!

$$C[1] = 1$$

for $i = 2$ to n do

initialization

$$\maxC = -1$$

$$\text{parent}[i] = \text{parent}$$

for $h = 1$ to $i - 1$ do

? { vedl pagine precedenti

$$C[i] = \maxC + 1$$

$$P[i] = \underline{\text{parent}}$$

Quanto costa?

{ for $i = 2$ to n do
costante
{ for $h = 1$ to $i - 1$ do
costante cost

$$\sum_{i=1}^n \text{cost} \cdot i = \frac{n(n+1)}{2} \approx n^2$$

costo di riempire C è

$$O(n^2)$$



costo per calcolare

$$d_n = |LIS(S)|$$

$$O(n^2) + O(n) = O(n^2)$$

vedere il
max valore
in C

• costo in tempo
quadratico

• costo in spazio è
lineare

memoria

